

Sistema de Geração de Provas

Versão do documento: 1.10

1. Visão do Produto

Plataforma web para professores e alunos, com app nativo **exclusivo para o professor** (usado na correção via câmera/QR Code), com foco em:

- Criação flexível de provas (quantidade livre de questões, até um limite de 20, e pontuação independente por questão).
- Geração de PDFs com controle fino sobre versões, embaralhamento e identificação do aluno.
- Correção via app mobile, exclusivo para o professor, usando QR Code + leitura do gabarito preenchido.
- Atribuição automática de nota quando a prova foi gerada **com identificação do aluno**.
- Lançamento manual de nota pelo professor quando a prova foi gerada **sem identificação**.

2. Personas

Persona	Descrição	Principais objetivos
Professor	Usuário autenticado	Criar provas, gerar PDFs personalizados, corrigir via app, lançar notas quando necessário, gerar relatórios
Aluno	Usuário autenticado	Consultar provas atribuídas e histórico de notas

3. Features e User Stories

Feature 1 – Cadastro e Autenticação (Professores e Alunos)

User Stories

- Como professor, quero me cadastrar e fazer login.
- Como aluno, quero me cadastrar e fazer login.
- Como usuário autenticado, quero consultar meus próprios dados de perfil.

Acceptance Criteria

- ☐ Cadastro separado por tipo de usuário (professor | estudante).
- ☐ Campos: nome completo, e-mail, senha (mín. 8 caracteres).
- ☐ E-mail único no sistema.
- ☐ E-mail do domínio @catolicasc.edu.br para alunos.
- ☐ E-mail do domínio @catolicasc.org.br para professores.
- ☐ Login retorna JWT + refresh token.
- ☐ Recuperação de senha por e-mail.
- ☐ Rate limiting contra força bruta.
- ☐ Logout e "sair de todos os dispositivos" (invalida todos os RefreshToken do usuário).
- ☐ Endpoint autenticado para o usuário consultar seus próprios dados (nome, e-mail, tipo).

Modelo resumido

```
// User (base para Professor e Aluno)
{
  id: string
  role: "professor" | "estudante"
  fullName: string
  email: string // único; domínio validado por role no cadastro
  passwordHash: string
  createdAt: Date
  anonymizedAt?: Date // preenchido quando a conta é anonimizada (ver 1.1 / LGPD)
}

// RefreshToken
{
  id: string
  userId: string
  tokenHash: string
  deviceInfo?: string // user-agent / identificador do dispositivo, para exibir em "sair de todos os dispositivos"
  issuedAt: Date
  expiresAt: Date
  revokedAt?: Date // preenchido em logout / logout-all / rotação
}
```

logout revoga apenas o RefreshToken do dispositivo atual (o presente na requisição). logout-all marca revokedAt em todos os RefreshToken ativos do userId.

Feature 1.1 – Anonimização de Conta (LGPD)

User Story Como usuário (professor ou aluno), quero poder anonimizar minha conta, para que meus dados pessoais deixem de estar associados às notas e correções já registradas no sistema.

Acceptance Criteria

- ☐ O usuário pode solicitar a anonimização da própria conta via `POST /me/anonymize`, autenticado.
- ☐ Ao anonimizar: `fullName` e `email` são substituídos por placeholders não reversíveis (ex: "Usuário removido", "anon-`<id>@removido.local`"); `passwordHash` é invalidado; todos os `RefreshToken`s do usuário são revogados (efeito equivalente a `logout-all`).
- ☐ `anonymizedAt` é preenchido com a data/hora da operação.
- ☐ **Não é exclusão real:** `Corrections`, notas, matrículas (`ClassEnrollment`) e demais registros vinculados ao `userId` são preservados, garantindo a integridade de relatórios, estatísticas e histórico de outras pessoas (ex: notas de uma turma não ficam incompletas).
- ☐ Uma conta anonimizada não pode mais fazer login.
- ☐ Um e-mail anonimizado libera o endereço de e-mail original para novo cadastro (já que o placeholder ocupa o lugar do e-mail antigo).
- ☐ A anonimização é irreversível.

Feature 2 – Gerenciamento de Questões (Professor)

User Story Como professor, quero criar, editar, listar e excluir questões objetivas e discursivas.

Acceptance Criteria

- ☐ Tipos: objetiva e discursiva.
- ☐ Questão objetiva: 2 a 5 alternativas, exatamente uma correta.
- ☐ Questão discursiva: enunciado + pontuação máxima.
- ☐ Suporte a Markdown básico.
- ☐ Tags.
- ☐ Soft-delete.
- ☐ Filtros por tipo, tags e texto.

Feature 3 – Criação de Turmas

User Story Como professor, quero criar turmas e matricular meus alunos nelas, para poder aplicar provas a um grupo específico de alunos.

Acceptance Criteria

- ☐ Posso criar uma turma informando nome, disciplina e período/ano letivo.
- ☐ Uma turma pertence a um único professor responsável.
- ☐ Toda turma nasce com um código de convite (`inviteCode`) único, gerado automaticamente na criação.
- ☐ Posso matricular alunos já cadastrados diretamente por e-mail.
- ☐ Posso visualizar o código de convite da turma a qualquer momento, para compartilhar com os alunos.
- ☐ Posso regenerar o código de convite (ex: em caso de vazamento), o que invalida imediatamente o código anterior.
- ☐ Um aluno pode se matricular sozinho informando o código de convite:
 - Se o e-mail informado já tiver conta cadastrada, o sistema apenas realiza a matrícula.
 - Se o e-mail ainda não tiver conta, o sistema cria a conta de aluno (validando domínio `@catolicasc.edu.br`) e realiza a matrícula na mesma operação, retornando o usuário já autenticado (JWT + refresh).
- ☐ Posso listar os alunos matriculados em uma turma.
- ☐ Posso remover um aluno de uma turma, sem apagar o histórico de notas já lançadas para ele.
- ☐ Posso editar nome, disciplina e período da turma.
- ☐ Posso arquivar (soft-delete) uma turma sem apagar o histórico de aplicações associadas a ela.
- ☐ Um aluno pode estar matriculado em mais de uma turma.
- ☐ Um aluno não pode se matricular duas vezes na mesma turma pelo código (se já matriculado, o endpoint apenas confirma o estado, sem duplicar `ClassEnrollment`).

Modelo resumido

```
// Class
{
  id: string
  teacherId: string
  name: string
```

```

subject: string          // disciplina
term: string             // ex: "2026/1"
status: "active" | "archived"
inviteCode: string       // único; regenerável pelo professor
createdAt: Date
}

// ClassEnrollment
{
  id: string
  classId: string
  studentId: string
  enrolledAt: Date
  status: "active" | "removed"
  enrolledVia: "teacher" | "invite_code" // rastreabilidade de como o aluno entrou
}

```

Feature 4 – Criação de Provas

User Story

Como professor, quero montar uma prova com quantas questões desejar, definindo o valor de cada questão de forma independente. A prova é um conteúdo reutilizável, que poderá ser aplicado a uma ou mais turmas posteriormente (ver Feature 5).

Acceptance Criteria

- ☐ Posso criar uma prova com título e descrição.
- ☐ Posso adicionar quantas questões quiser (objetivas e/ou discursivas), até o limite de 20.
- ☐ Posso definir a **pontuação de cada questão individualmente** (não precisa ser uniforme).
- ☐ **A soma das pontuações não é validada pelo sistema.** A responsabilidade por garantir a pontuação total desejada (ex: somar 10,0) é exclusiva do professor.

- ☐ Posso definir a ordem inicial das questões.
- ☐ A prova fica no estado `draft` (`draft`) até ser aplicada a uma turma pela primeira vez.
- ☐ Ao criar a primeira `Application` referenciando este `Exam` (`POST /applications`), o status muda automaticamente para `ready` . Não há endpoint dedicado para essa transição — é efeito colateral da criação da Aplicação.
- ☐ O professor pode arquivar uma prova manualmente (`closed`), a qualquer momento, via `PATCH /exams/:id/archive` .
- ☐ Uma prova `closed` **não pode** ser usada para criar novas `Application` s. `Application` s já existentes que a referenciam continuam funcionando normalmente (PDF, correção, relatórios).
- ☐ Provas `closed` continuam listadas (com filtro `?status=draft|ready|closed`) — arquivar não é exclusão, apenas oculta da seleção ao criar novas aplicações.
- ☐ Uma prova pertence a um único professor e **não está vinculada a nenhuma turma diretamente** — a mesma prova pode ser aplicada a quantas turmas o professor desejar, ou reaplicada à mesma turma (ex: segunda chamada), através de Aplicações independentes (Feature 5).

Modelo resumido

```
{
  id: string
  teacherId: string
  title: string
  description?: string
  questions: {
    questionId: string
    order: number
    score: number           // pontuação individual, sem validação de soma
  }[]
  status: "draft" | "ready" | "closed" // draft→ready: automático na 1ª Application; ready/draft→closed: manual (archive)
  createdAt: Date
}
```

Feature 5 – Criação de Aplicação

User Story

Como professor, quero aplicar uma prova já criada a uma turma específica, para então gerar o PDF, distribuir a prova e realizar a correção referente àquela aplicação.

Acceptance Criteria

- ☐ Posso criar uma **Aplicação** escolhendo uma prova (Exam) já existente e uma turma de destino.
- ☐ A mesma prova pode ser aplicada a múltiplas turmas (uma Aplicação por turma) ou reaplicada à mesma turma (ex: segunda chamada, reaplicação), gerando Aplicações independentes entre si.
- ☐ A Aplicação nasce no estado `rascunho` até que o professor gere o PDF/versões (Feature 6).
- ☐ Uma Aplicação pertence a um único professor e está associada a exatamente uma prova e uma turma.
- ☐ Todas as configurações de geração de PDF (versões, embaralhamento, identificação), publicação de gabarito, correções e notas passam a ser referentes à **Aplicação** — não mais à prova (Exam) diretamente.

Modelo resumido

```
// Application
{
  id: string
  examId: string
  classId: string
  teacherId: string
  status: "draft" | "generated" | "closed"
  createdAt: Date
}
```

Feature 6 – Geração de PDF e Versões

User Story

Como professor, quero gerar o PDF de uma aplicação para impressão, com controle total sobre versões, embaralhamento e identificação dos alunos.

Acceptance Criteria

O professor, após criar a aplicação, configura a geração do PDF com as seguintes opções:

1. Quantidade de versões

- Pode escolher gerar 1 ou mais versões da prova.

2. Embaralhamento (por versão)

- Embaralhar ou não a **ordem das questões**.
- Embaralhar ou não a **ordem das alternativas** das questões objetivas.
- As opções de embaralhamento podem ser diferentes entre as versões (ou iguais).

3. Identificação do aluno

- **Com identificação:** o nome (e opcionalmente matrícula) do aluno é impresso no cabeçalho da prova. Cada aluno recebe uma versão específica.
- **Sem identificação:** a prova é anônima no cabeçalho (sem nome pré-impresso do aluno). O aluno escreve seu nome e matrícula à mão na prova impressa, e o professor precisará lançar a nota manualmente depois da correção (ver Feature 9).

Regras importantes

- Quando a opção **“com identificação”** é escolhida, o sistema deve:
 - Associar cada aluno matriculado na turma da aplicação a uma versão específica.
 - Gerar um QR Code único por aluno que contenha as informações necessárias para a correção (versão + aluno).
- Quando a opção **“sem identificação”** é escolhida:

- O QR Code identifica apenas a **versão** da prova.
- A correção pelo app calcula a nota, mas **não atribui automaticamente** ao aluno. O professor identifica o aluno pelo nome/matricula escritos à mão na prova e lança a nota depois (Feature 9).
- **O PDF gerado é um único arquivo consolidado**, contendo em sequência todas as provas impressas da aplicação (todas as versões e, quando houver identificação, uma prova por aluno). Não são gerados arquivos separados por aluno ou por versão.
- **A ordem de embaralhamento é materializada e persistida por versão.** As flags `shuffleQuestions` / `shuffleAlternatives` controlam apenas se o embaralhamento acontece; o resultado efetivo (qual questão aparece em qual posição, e qual alternativa impressa como "A", "B", "C"... corresponde a qual `alternativeId` da questão original) é gravado em `layout` no momento da geração. Isso é obrigatório para que a correção via app saiba interpretar corretamente qual letra marcada corresponde à alternativa correta.

Regeneração do PDF

- ☐ O professor pode gerar o PDF novamente para a mesma Aplicação, alterando quantidade de versões, embaralhamento ou identificação.
- ☐ **Regeneração só é permitida enquanto não existir nenhuma `Correction` confirmada para a Aplicação.** Caso já exista, o endpoint retorna erro orientando o professor a criar uma nova Aplicação (ex: segunda chamada) em vez de regenerar.
- ☐ Ao regenerar com sucesso, todas as `ExamVersion`s (e `ExamAssignment`s) anteriores daquela Aplicação são substituídas — os QR Codes antigos deixam de ser válidos.
- ☐ O `pdfUrl` da Aplicação é sobrescrito a cada regeneração.

Modelo de Versão / Impressão

```
// ExamVersion
{
  id: string
  applicationId: string
  versionNumber: number
  shuffleQuestions: boolean
  shuffleAlternatives: boolean
  withStudentIdentification: boolean
  layout: { // ordem efetivamente impressa, materializada na geração
```

```

    questionOrder: string[]           // questionIds, na ordem impressa nesta versão
    alternativeOrder: {               // apenas para questões objetivas
      questionId: string
      printedOrder: string[]         // alternativeIds, na ordem/letras impressas (A, B, C...)
    }[]
  }
  answerKeyPublished: boolean
  answerKeyPublishedAt?: Date
  publicCode: string                 // código de acesso público ao gabarito, distinto do qrCodePayload
  qrCodePayload: string              // uso exclusivo do app do professor; nunca exposto por rota pública
  createdAt: Date
}

// ExamAssignment (somente quando withStudentIdentification = true)
{
  id: string
  examVersionId: string
  studentId: string
  qrCodePayload: string              // dados específicos do aluno para o app ler
}

```

O campo `pdfUrl` fica registrado na **Aplicação** (Feature 5), não em cada versão individualmente, já que se trata de um único arquivo consolidado compartilhado por todas as versões daquela aplicação.

`qrCodePayload` (versão ou aluno) é de uso exclusivo do app mobile do professor. `publicCode` é um identificador separado, usado apenas pela rota pública de consulta de gabarito (`GET /public/answer-key/:publicCode`), para não expor o mesmo código usado na correção.

Feature 7 – Publicação de Gabarito

User Story Como professor, quero publicar o gabarito de uma aplicação.

Acceptance Criteria

- ☐ Posso publicar o gabarito para toda a aplicação ou por versão.
- ☐ Alunos só conseguem ver o gabarito após a publicação.
- ☐ Rota pública (ou autenticada para alunos) para consulta.

Feature 8 – Correção via App Mobile

User Story

Como professor, quero corrigir uma prova usando o app mobile, lendo o QR Code e o gabarito preenchido pelo aluno, e ter o sistema calcular a nota automaticamente. A atribuição da nota ao aluno depende de a prova ter sido gerada com identificação. **O acesso à leitura de QR Code existe exclusivamente no app mobile do professor** (não há leitura de QR Code na web).

Acceptance Criteria

8.1 Leitura inicial

- ☐ O app lê o **QR Code** impresso na prova.
- ☐ A partir do QR Code, o app identifica:
 - o A versão da prova
 - o (Se existir) o aluno correspondente
- ☐ O app chama a API e recupera o **gabarito oficial** daquela versão (ou usa a cópia local, se offline — ver 8.6).
- ☐ Ao abrir uma Aplicação para corrigir (enquanto há conexão), o app baixa antecipadamente e armazena localmente o gabarito oficial de **todas as versões** daquela Aplicação (GET /mobile/applications/:id/answer-keys), permitindo corrigir depois mesmo sem sinal.
- ☐ Durante a correção, o app tenta buscar o gabarito atualizado da API primeiro; se não houver conexão, usa a última cópia baixada localmente para aquela versão.
- ☐ Se não houver conexão e nenhuma cópia local tiver sido baixada anteriormente para aquela versão, a correção **não pode ser processada** — o app avisa o professor e não permite confirmar (evita nota calculada contra gabarito ausente ou desatualizado).

8.2 Leitura do gabarito preenchido

- ☐ O professor aponta a câmera para o cartão-resposta (quadros ou círculos preenchidos com caneta).
- ☐ O app detecta e interpreta as marcações.
- ☐ O app confronta as respostas lidas com o gabarito oficial.
- ☐ Mostra ao professor, questão por questão, se o aluno **acertou ou errou**.
- ☐ Calcula a **nota total** automaticamente (respeitando a pontuação individual de cada questão).
- ☐ Permite que o professor corrija manualmente alguma leitura antes de confirmar.

8.3 Questões discursivas

- ☐ Após a leitura objetiva, o professor informa as notas das questões discursivas.
- ☐ A nota final é recalculada.

8.4 Identificação do aluno (provas sem identificação impressa)

- ☐ Quando a prova foi gerada **sem identificação**, o aluno escreve nome e matrícula à mão na prova física.
- ☐ No momento da confirmação da correção, o app permite (opcionalmente) que o professor digite o nome/matricula lidos na prova física, para pré-preencher o lançamento manual da nota (Feature 9).
- ☐ Esse preenchimento é apenas informativo/auxiliar: **não** provoca atribuição automática da nota — o lançamento continua sendo um passo explícito do professor (Feature 9).

8.5 Confirmação e registro da nota

- ☐ Ao confirmar a correção:
 - o **Se a prova foi gerada COM identificação do aluno** → a nota é **registrada automaticamente** via API e atribuída ao aluno assim que houver sincronização com o servidor (imediata, se online; em segundo plano, assim que a conexão retornar — ver 8.6).
 - o **Se a prova foi gerada SEM identificação do aluno** → a nota é apenas calculada e exibida (com nome/matricula opcionalmente anotados, ver 8.4). O professor deverá lançar manualmente a nota para o aluno posteriormente.

8.6 Sincronização Offline (Fila de Correções)

O app de correção precisa funcionar de forma confiável mesmo sem conexão com a internet no momento da correção. Estratégia:

- ☐ Toda correção realizada no app (leitura de QR Code + gabarito + confirmação) é salva **primeiro no armazenamento local do dispositivo** (fila local), independentemente de haver conexão com a internet no momento.
- ☐ Cada item da fila local recebe um identificador único gerado no próprio dispositivo (`clientCorrectionId`), usado posteriormente para deduplicação no servidor.
- ☐ A correção local guarda também uma cópia do gabarito usado no momento (`answerKeySnapshot`), garantindo que o cálculo da nota permaneça consistente mesmo que o gabarito oficial seja alterado antes da sincronização.
- ☐ Quando o dispositivo detecta conexão com a internet, o app tenta sincronizar automaticamente os itens pendentes da fila, em segundo plano, sem bloquear o uso do app.
- ☐ O professor pode visualizar o status de cada correção na fila: `pendente de sincronização` , `sincronizada` OU `erro de sincronização` .
- ☐ O professor pode forçar manualmente uma nova tentativa de sincronização.
- ☐ O endpoint de sincronização aceita envio em **lote (batch)** das correções pendentes.
- ☐ **Deduplicação:** se uma correção com o mesmo `clientCorrectionId` já foi recebida pelo servidor, o servidor descarta o reenvio e retorna sucesso (operação idempotente).
- ☐ **Tratamento de conflitos:**
 - Se o gabarito da versão foi alterado/republicado no servidor entre o momento da correção local e a sincronização, o servidor recalcula a nota usando o `answerKeySnapshot` salvo junto com a correção original, e sinaliza ao professor que houve essa situação.
 - Se duas correções chegarem para a mesma versão + mesmo aluno em uma prova **com identificação** (ex: o mesmo professor corrigiu duas vezes, ou dois dispositivos), o servidor **mantém a primeira nota registrada** e sinaliza um conflito para revisão manual — a nota não é sobrescrita silenciosamente.
- ☐ Correções que falharem na sincronização por erro de validação (ex: versão inexistente, gabarito não encontrado) ficam marcadas como erro, e o professor é notificado, podendo corrigir os dados e reenviar.
- ☐ A fila local persiste mesmo se o app for fechado ou reiniciado.

Modelo de fila local (dispositivo)

```
{
  clientCorrectionId: string    // UUID gerado no dispositivo
  examVersionId: string
  studentId?: string
  reportedStudentName?: string // lido na prova física, quando sem identificação
  reportedStudentRegistration?: string // idem
  objectiveAnswers: { questionId: string; selectedAlternativeId: string }[]
  discursiveScores: { questionId: string; score: number }[]
  answerKeySnapshot: object     // gabarito usado no momento da correção
  capturedAt: Date              // quando a correção foi feita no dispositivo
  syncStatus: "pending" | "synced" | "error"
  syncError?: string
}
```

8.7 Requisitos técnicos

- ☐ Leitura do gabarito e cálculo da nota funcionam **totalmente offline** após o download prévio do gabarito da versão.
- ☐ Feedback visual claro durante a leitura (área detectada, qualidade da imagem).
- ☐ Taxa de erro de leitura < 1% em condições normais de uso.
- ☐ Tolerância a variações de iluminação, rotação e leve distorção de perspectiva.

Feature 9 – Lançamento Manual de Nota

User Story

Como professor, quando a prova foi gerada **sem identificação do aluno**, quero associar a nota já calculada pelo app ao aluno correspondente, identificando-o pelo nome/matricula que ele escreveu à mão na prova física.

Acceptance Criteria

- ☐ Após a correção pelo app (modo sem identificação), a nota calculada fica registrada como uma `Correction` **sem `studentId`** (pendente de atribuição).
- ☐ Posso listar, por aplicação, todas as `Correction`s pendentes de atribuição (sem `studentId`).
- ☐ Se o professor já informou nome/matricula durante a correção no app (Feature 8.4), esses dados vêm **pré-preenchidos** na listagem para agilizar a busca do aluno correto na turma.
- ☐ Na web (ou no app), o professor seleciona uma `Correction` pendente e associa-a a um aluno da turma (buscando pelo nome/matricula).
- ☐ **A atribuição preenche o `studentId` na `Correction` já existente — não cria um novo registro de nota.** Isso evita contagem duplicada em relatórios e exportações.
- ☐ É possível informar também observações no momento da atribuição.
- ☐ Uma vez atribuída, a nota passa a aparecer no histórico do aluno da mesma forma que as notas automáticas.
- ☐ Não é possível atribuir duas `Correction`s da mesma versão ao mesmo aluno (evita duplicidade de nota para um mesmo aluno em uma aplicação).

Feature 10 – Relatórios de Notas (Professor)

User Story Como professor, quero gerar relatórios de notas de uma aplicação ou de várias aplicações.

Acceptance Criteria

- ☐ Relatório por Aplicação: lista de alunos, notas, média, mediana, desvio padrão, distribuição.
- ☐ Relatório consolidado (várias aplicações).
- ☐ Filtros por turma, disciplina, período.
- ☐ Exportação em CSV, Excel e PDF.
- ☐ Visualização de estatísticas básicas (gráficos simples).

Feature 11 – Histórico de Notas do Aluno

User Story Como aluno, quero consultar meu histórico completo de notas.

Acceptance Criteria

- ☐ Lista de todas as provas que realizei, com nota, data e professor.
- ☐ Detalhamento de cada prova (acertos por questão, quando o gabarito estiver publicado).
- ☐ Filtros por disciplina / período.
- ☐ Visualização de evolução (gráfico simples de desempenho ao longo do tempo).
- ☐ Acesso apenas às próprias notas (isolamento total).

4. Fluxo Resumido

Fluxo com identificação do aluno

1. Professor cria/edita a turma e matricula os alunos (Feature 3).
2. Professor cria a prova (banco de questões) (Feature 4).
3. Professor cria uma Aplicação, associando a prova a uma turma (Feature 5).
4. Gera o PDF (arquivo único consolidado), escolhendo:
 - o Quantidade de versões
 - o Embaralhamento de questões e/ou alternativas
 - o **Com identificação do aluno**
5. Sistema atribui uma versão + QR Code único para cada aluno.
6. Professor imprime o PDF consolidado.
7. No app mobile do professor (única superfície com acesso à leitura de QR Code), lê o QR Code → lê o gabarito preenchido → calcula a nota.
Se estiver offline, a correção é salva na fila local do dispositivo.

8. Ao confirmar, a nota é **automaticamente registrada** para o aluno assim que houver sincronização com o servidor (imediate, se online; em segundo plano, assim que a conexão retornar).

Fluxo sem identificação do aluno

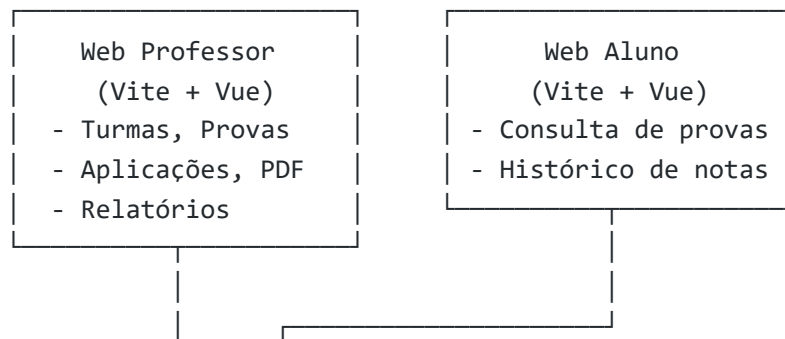
1. Professor cria/edita a turma e matricula os alunos.
2. Professor cria a prova.
3. Professor cria a Aplicação, associando a prova a uma turma.
4. Gera o PDF (arquivo único consolidado), escolhendo:
 - o Quantidade de versões
 - o Embaralhamento
 - o **Sem identificação do aluno**
5. Alunos escrevem nome e matrícula à mão na prova impressa.
6. QR Code identifica apenas a versão.
7. No app mobile do professor, lê o QR Code → lê o gabarito → calcula a nota (podendo informar o nome/matricula lidos na prova). Se offline, vai para a fila local do dispositivo.
8. Ao confirmar, a nota **não** é atribuída automaticamente.
9. Professor lança manualmente a nota para o aluno depois, usando o nome/matricula escritos na prova para identificá-lo corretamente (Feature 9).

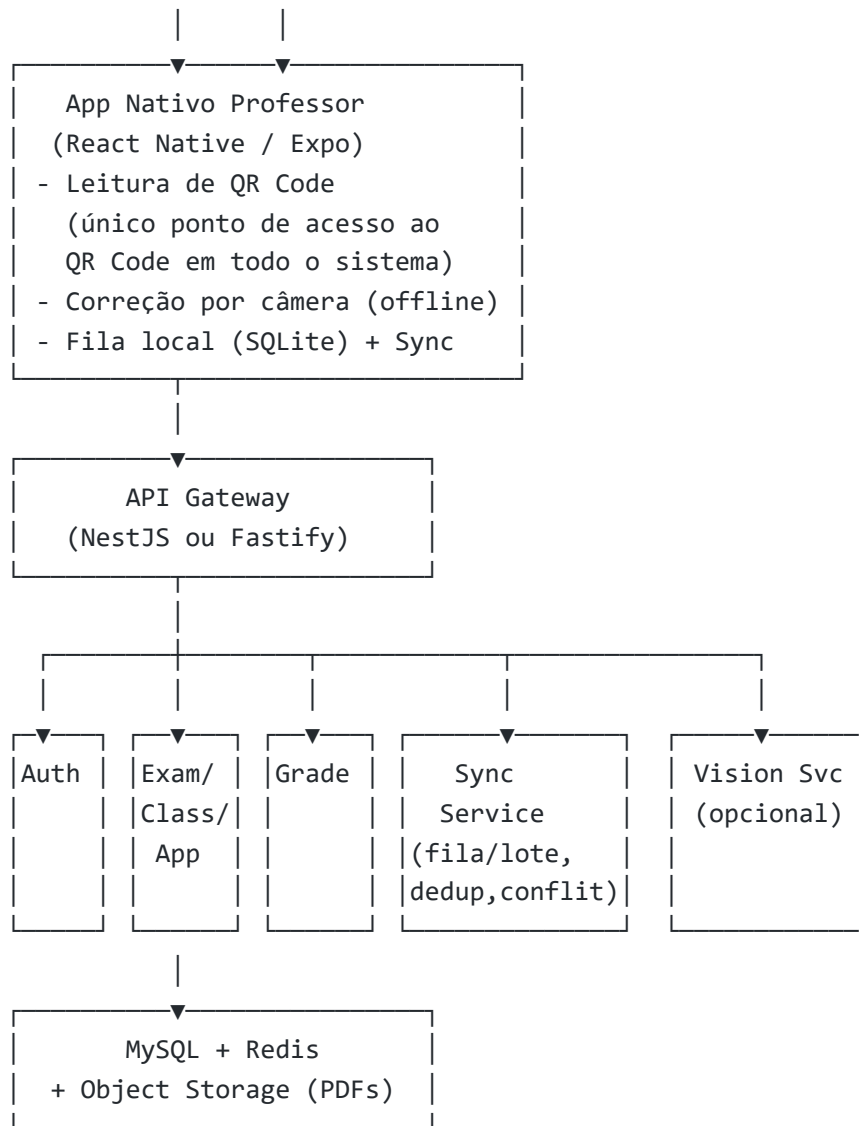
5. Requisitos Não-Funcionais (Produção)

Categoria	Requisito
Escalabilidade	500-600 professores + 10.000 alunos ativos no primeiro ano

Categoria	Requisito
Performance	API p95 < 300ms para operações comuns
Disponibilidade	99,5% uptime mensal
Segurança	HTTPS, JWT + refresh, rate limiting, isolamento total entre alunos
Privacidade (LGPD)	Aluno só acessa seus próprios dados. Logs de acesso a notas. Anonimização de conta disponível sob demanda (Feature 1.1); exclusão física de dados fica fora do MVP
Offline	App de correção funciona totalmente offline (fila local com sincronização automática em segundo plano ao reconectar)
Observabilidade	Logs estruturados + métricas + alertas
Backup	Backup diário do banco com retenção de 30 dias
Testes	Cobertura ≥ 80% nas regras críticas de negócio

6. Arquitetura de Alto Nível





Vision Svc (opcional): a leitura do cartão-resposta ocorre **no próprio dispositivo** (on-device), o que permite funcionamento 100% offline (Feature 8.7). O Vision Svc server-side é um componente opcional para fallback/verificação de leitura com maior precisão quando há conectividade — não é um pré-requisito para o fluxo principal de correção.

Sync Service: componente responsável por receber lotes de correções da fila local dos apps, aplicar deduplicação por `clientCorrectionId` e resolver conflitos conforme as regras da Feature 8.6.

7. Endpoints Principais

Método	Rota	Descrição	Auth
POST	/auth/register	Cadastro (teacher ou student)	Público
POST	/auth/login	Login	Público
POST	/auth/refresh	Renovar token	Refresh
POST	/auth/logout	Logout do dispositivo atual	Autenticado
POST	/auth/logout-all	Logout de todos os dispositivos (invalida refresh tokens)	Autenticado
POST	/auth/forgot-password	Solicita e-mail de recuperação de senha	Público
POST	/auth/reset-password	Define nova senha a partir do token recebido por e-mail	Público
GET	/me	Dados do usuário autenticado (nome, e-mail, tipo)	Autenticado
POST	/me/anonymize	Anonimiza a própria conta (substitui nome/e-mail por placeholders); preserva <code>Correction s</code> e notas já lançadas	Autenticado
GET	/questions	Listar questões	Professor
POST	/questions	Criar questão	Professor
PATCH	/questions/:id	Editar questão	Professor
DELETE	/questions/:id	Excluir questão (soft-delete)	Professor

Método	Rota	Descrição	Auth
POST	/classes	Criar turma	Professor
GET	/classes	Listar turmas	Professor
PATCH	/classes/:id	Editar turma (nome, disciplina, período)	Professor
PATCH	/classes/:id/archive	Arquivar turma (soft-delete)	Professor
POST	/classes/:id/students	Matricular aluno já cadastrado na turma (por e-mail)	Professor
GET	/classes/:id/students	Listar alunos matriculados na turma	Professor
DELETE	/classes/:id/students/:studentId	Remover aluno da turma	Professor
GET	/classes/:id/invite-code	Consultar o código de convite atual da turma	Professor
POST	/classes/:id/invite-code/regenerate	Regenerar o código de convite (invalida o anterior)	Professor
POST	/classes/join-by-code	Aluno se matricula via código de convite (cria conta se e-mail ainda não existir, ou apenas matricula)	Público
POST	/exams	Criar prova (banco de questões, sem vínculo com turma)	Professor
GET	/exams	Listar provas do professor	Professor
GET	/exams/:id	Detalhar prova (com lista de questões)	Professor
PATCH	/exams/:id	Editar prova (título, descrição, questões, pontuações)	Professor
PATCH	/exams/:id/archive	Arquivar prova (status → closed); bloqueia novas aplicações, preserva aplicações existentes	Professor
POST	/applications	Criar aplicação (associa uma prova a uma turma)	Professor

Método	Rota	Descrição	Auth
GET	/applications	Listar aplicações do professor	Professor
GET	/applications/:id	Detalhar aplicação	Professor
POST	/applications/:id/generate-pdf	Gerar (ou regenerar) versões + PDF único consolidado. Regeneração é bloqueada se já houver <code>Correction</code> confirmada	Professor
GET	/applications/:id/versions	Listar versões geradas	Professor
GET	/applications/:id/students	Listar alunos e versões atribuídas	Professor
POST	/applications/:id/versions/:versionId/publish-answer-key	Publicar gabarito de uma versão específica	Professor
POST	/applications/:id/publish-answer-key	Publicar gabarito de toda a aplicação (todas as versões)	Professor
GET	/applications/:id/corrections	Listar correções da aplicação (com filtro <code>?assigned=false</code> para pendentes de atribuição)	Professor
PATCH	/applications/:id/corrections/:correctionId	Atribuir uma <code>Correction</code> pendente a um aluno (lançamento manual — Feature 9)	Professor
GET	/applications/:id/report	Relatório de notas da aplicação	Professor
GET	/reports/consolidated	Relatório consolidado (aceita <code>?format=csv xlsx pdf</code>)	Professor
GET	/student/exams	Provas atribuídas ao aluno	Aluno
GET	/student/grades	Histórico completo de notas	Aluno
GET	/student/grades/:applicationId	Detalhe de uma prova específica	Aluno

Método	Rota	Descrição	Auth
GET	/public/answer-key/:publicCode	Gabarito (quando publicado) — usa <code>publicCode</code> da versão, distinto do QR Code do app	Público/Aluno
GET	/mobile/applications/:id/answer-keys	Baixa o gabarito oficial de todas as versões da aplicação, para cache local antecipado (uso offline)	Professor
POST	/mobile/corrections/scan	App envia dados do QR Code + respostas lidas → retorna acertos e nota (uso online)	Professor
POST	/mobile/corrections/confirm	Confirma a correção. Se tiver identificação, registra a nota automaticamente (online)	Professor
POST	/mobile/corrections/sync	Sincroniza em lote as correções da fila local (idempotente por <code>clientCorrectionId</code> , trata conflitos)	Professor

8. Modelo de Dados – Principais Entidades

```
// User (base para Professor e Aluno)
{
  id: string
  role: "professor" | "estudante"
  fullName: string
  email: string // único; domínio validado por role no cadastro
  passwordHash: string
  createdAt: Date
  anonymizedAt?: Date // preenchido quando a conta é anonimizada (ver Feature 1.1 / LGPD)
}
```



```
// RefreshToken
{
  id: string
  userId: string
  tokenHash: string
  deviceInfo?: string
  issuedAt: Date
  expiresAt: Date
  revokedAt?: Date
}

// Class
{
  id: string
  teacherId: string
  name: string
  subject: string
  term: string
  status: "active" | "archived"
  inviteCode: string // único; regenerável pelo professor
}

// ClassEnrollment
{
  id: string
  classId: string
  studentId: string
  status: "active" | "removed"
  enrolledVia: "teacher" | "invite_code"
}

// Question
{
  id: string
  teacherId: string
```

```

type: "objetiva" | "discursiva"
statement: string           // enunciado, suporta Markdown básico
tags: string[]
// Se type = "objetiva"
alternatives?: { id: string; text: string }[] // 2 a 5 alternativas
correctAlternativeId?: string
// Se type = "discursiva"
maxScore?: number
deletedAt?: Date           // soft-delete
}

// Exam
{
  id: string
  teacherId: string
  title: string
  questions: { questionId: string; order: number; score: number }[]
  status: "draft" | "ready" | "closed" // draft→ready: automático na 1ª Application; qualquer→closed: manual (archive)
}

// Application (prova aplicada a uma turma)
{
  id: string
  examId: string
  classId: string
  teacherId: string
  status: "draft" | "generated" | "closed" // "closed" reservado; sem endpoint de transição no MVP (ver Feature 5)
  pdfUrl?: string // PDF único consolidado desta aplicação; sobrescrito a cada regeneração
}

// ExamVersion
{
  id: string
  applicationId: string
  versionNumber: number
}

```

```

shuffleQuestions: boolean
shuffleAlternatives: boolean
withStudentIdentification: boolean
layout: {
    questionOrder: string[]
    alternativeOrder: {
        questionId: string
        printedOrder: string[]
    }
}
answerKeyPublished: boolean
answerKeyPublishedAt?: Date
publicCode: string
qrCodePayload: string

// ordem efetivamente impressa, materializada na geração/regeneração
// alternativeIds, na ordem/letras impressas
// acesso público ao gabarito, distinto do qrCodePayload
// uso exclusivo do app do professor

}

// ExamAssignment (somente quando withStudentIdentification = true)
{
    id: string
    examVersionId: string
    studentId: string
    qrCodePayload: string
}

// Correction
{
    id: string
    examVersionId: string
    studentId?: string
    reportedStudentName?: string
    reportedStudentRegistration?: string
    objectiveResults: { questionId: string; correct: boolean; score: number }[]
    discursiveScores: { questionId: string; score: number }[]
    totalScore: number
    notes?: string
    // preenchido automaticamente se houver identificação, ou depois via PATCH /applications/:id/
    // lido na prova física, quando sem identificação
    // idem
}

```

```

confirmedAt: Date
correctedBy: string // teacherId
isAutomaticallyAssigned: boolean // true se veio de prova com identificação
clientCorrectionId: string // origem: fila local do app, usado para deduplicação
syncStatus: "pending" | "synced" | "error"
}

```

9. Roadmap Atualizado

Fase	Foco	Entrega principal
1	Fundação	Auth (professor + aluno) + CRUD de questões + Turmas
2	Provas e Aplicações	Criação de provas (banco de questões) + criação de aplicações (prova x turma)
3	Geração de PDF e Versões	PDF único consolidado, quantidade de versões, embaralhamento, identificação do aluno
4	Backend de correção	Endpoint de scan/confirm/sync + lógica de atribuição automática vs manual + deduplicação/conflitos
5	App nativo	Leitura de QR Code (exclusiva do app professor) + leitura offline do gabarito + fila local + sync
6	Lançamento manual	Interface para o professor lançar notas de aplicações sem identificação
7	Área do Aluno + Relatórios	Histórico e relatórios funcionando com os dois tipos de nota
8	Produção	Observabilidade, segurança, CI/CD, LGPD, escala

10. Critérios de “Pronto”

Uma feature só é considerada implementada quando:

- ☐ Todos os Acceptance Criteria estão passando.
- ☐ Testes automatizados cobrem os fluxos principais e de erro.
- ☐ Isolamento de dados entre alunos está garantido.
- ☐ Documentação OpenAPI atualizada.
- ☐ Logs e métricas instrumentados (quando aplicável).
- ☐ É possível gerar o PDF único consolidado, com e sem identificação do aluno.
- ☐ Embaralhamento de questões e de alternativas funciona de forma independente.
- ☐ O QR Code carrega corretamente as informações necessárias (versão $\pm$ aluno) e só é acessível pelo app mobile do professor.
- ☐ A confirmação da correção no app registra a nota automaticamente **somente** quando há identificação.
- ☐ Aplicações sem identificação exigem lançamento manual posterior, com apoio dos dados de nome/matricula lidos na prova física.
- ☐ A fila local de correções sincroniza corretamente, com deduplicação por `clientIdCorrectionId` e tratamento explícito de conflitos (gabarito alterado / nota já registrada).
- ☐ O app baixa e mantém em cache local o gabarito de uma aplicação antes de ir a campo, e recorre a essa cópia automaticamente quando não há conexão no momento da correção, sem permitir confirmar correções sem gabarito disponível (online ou em cache).